

CS5998 - Capstone Project – Final Project Report

**Machine Learning–Based Anomaly Detection
for
Telecom Network Performance Data Ingestion Pipelines**

*Gunawardana G.S.D.H.
258726T*

Abstract

Engineering Team of Dialog Axiata PLC operates large-scale telecom analytics and data engineering platforms that ingest Performance Management (PM) data from multiple Network Management Systems (NMS) such as Huawei U2020 and Ericsson ENM into centralized analytical environments based on Microsoft Fabric. These datasets are extensively used for operational monitoring, capacity planning, fault management, service assurance, anomaly analysis, and strategic business decision-making across the organization. Due to the scale and complexity of data fetching, ingestion pipelines are vulnerable to several operational failures including connectivity outages, delayed or partial file transfers, missing reporting intervals, and abnormal KPI values generated by upstream system failures.

When such failures occur, incomplete or corrupted datasets propagate into downstream analytics systems, resulting in misleading dashboards, inaccurate operational insights, and unreliable engineering and business decisions. Existing detection mechanisms within the data pipelines are largely rule-based and dependent on static threshold validation, row-count checks, or manual verification processes. These approaches are difficult to scale, require continuous operational maintenance, and often fail to adapt effectively to evolving network conditions.

This capstone project proposes and implements a Machine Learning (ML)-driven anomaly detection framework for telecom PM data ingestion pipelines within the operational context of Dialog Axiata PLC. The system utilizes real-world hourly telecom traffic datasets and evaluates multiple forecasting approaches including XGBoost, Facebook Prophet, NeuralProphet, and Long Short-Term Memory (LSTM) to learn normal traffic behavior and automatically identify anomalous patterns that may indicate ingestion or upstream system failures.

The developed framework performs data preprocessing, feature engineering, ML based forecasting, prediction validation, and anomaly detection using deviation analysis between actual and predicted traffic values. Anomalies are identified using configurable deviation thresholds together with null-value detection mechanisms to support early fault identification and operational troubleshooting.

Experimental evaluation demonstrates that ML-based forecasting models significantly improve anomaly detection capability compared to conventional rule-based approaches. Comparative analysis of the evaluated models highlights the strengths and limitations of each forecasting technique in telecom traffic prediction and ingestion anomaly detection applications.

The final system demonstrates the feasibility of integrating AI-driven anomaly detection into large-scale telecom data engineering environments at Dialog Axiata PLC to improve operational visibility, enhance data reliability, enable proactive fault detection, and support more accurate analytics-driven decision-making.

Contents

1	Introduction	1
2	Background and Motivation.....	2
3	Problem Statement	2
4	Project Objectives	3
5	Literature Review	3
5.1	Telecom Data Forecasting Models.....	3
5.2	Machine Learning-Based Forecasting.....	4
5.3	Deep Learning Forecasting	5
5.4	Prophet and NeuralProphet	5
5.5	Anomaly Detection in Telecom Systems.....	6
6	Dataset Description.....	7
6.1	Dataset Source.....	7
6.2	Dataset Structure.....	7
7	System Architecture	8
7.1	Data Ingestion.....	8
7.2	Data Preprocessing Layer.....	9
7.3	Feature Engineering Layer	9
7.4	Machine Learning / Forecasting Layer.....	9
7.5	Anomaly Detection Layer	10
7.6	Visualization and Anomaly File Generation Layer	10
8	Methodology.....	10
8.1	Data Collection	11
8.2	Datetime Construction	11
8.3	Feature Engineering.....	11
8.4	Training and Validation Split	12
8.5	Machine Learning / Forecasting	13
8.6	Anomaly Detection	13
9	Comparison of Machine Learning Models.....	14
9.1	XGBoost-Based Forecasting Model.....	15
9.1.1	Advantages	15
9.1.2	Model Configuration	16
9.1.3	Forecasting Strategy	16
9.1.4	Performance.....	16

9.2	Prophet Forecasting Model	16
9.2.1	Advantages	17
9.2.2	Limitations	17
9.2.3	Performance	18
9.3	LSTM Forecasting Model	18
9.3.1	Sequence Learning	18
9.3.2	Model Architecture	19
9.3.3	Hyperparameters	19
9.3.4	Advantages	19
9.3.5	Limitations	19
10	Anomaly Detection Framework	21
10.1	Detection Principle	21
10.2	Deviation Calculation	21
10.3	Threshold-Based Detection	22
10.4	Null Detection	22
10.5	Operational Relevance	22
11	Experimental Setup	23
11.1	Development Environment	23
11.2	Hardware	24
11.3	Evaluation Metrics	24
12	Results and Evaluation	25
12.1	Forecasting Accuracy Evaluation	25
12.2	Anomaly Detection Results	25
12.3	Visual Analysis	26
13	Comparative Analysis	26
14	Visualization and Interpretation	26
14.1	Traffic Pattern Visualization	26
14.2	Prediction Comparison	27
14.3	Deviation Visualization	27
14.4	Anomaly Highlighting	27
15	Discussion	27
16	Limitations	27
	Annex 1 - Reproducibility Checklist	29

1 Introduction

Telecommunication operators such as Dialog Axiata PLC nowadays rely heavily on centralized analytics and data engineering platforms to monitor network performance, optimize operational efficiency, and support strategic decision-making. Within modern telecom environments, Network Performance Management (PM) systems continuously generate large volumes of Key Performance Indicator (KPI) measurements related to traffic throughput, connected users, network availability, quality of service, and resource utilization across LTE and 5G infrastructures.

At Dialog Axiata PLC, these measurements are collected from multiple Network Management Systems (NMS) such as Huawei U2020 and Ericsson ENM and are transported through large-scale ingestion pipelines into centralized analytics platforms including enterprise data warehouses and cloud-based analytics environments. These datasets play a critical role in several operational and business functions, including:

- Network Operations Center (NOC) monitoring dashboards
- Capacity planning and network optimization
- Fault and incident management systems
- AI-driven analytics and automation platforms
- KPI reporting and regulatory compliance systems
- Executive and strategic decision-support platforms

In modern telecom operations, ingestion pipelines process extremely large volumes of hourly and daily PM data generated from thousands of network entities. Any failures occurring during the ingestion stage can significantly impact downstream analytical systems and operational visibility. Common ingestion-related issues include delayed file transfers, missing KPI batches, corrupted PM files and abnormal traffic fluctuations.

Traditional anomaly detection mechanisms used within telecom environments are predominantly rule-based and depend heavily on manually configured static thresholds. Although such approaches are simple to implement, they often fail to adapt to dynamic traffic patterns, seasonal variations, and rapidly changing network conditions observed in real-world telecom systems.

The primary challenge addressed in this project is the automated detection of abnormal telecom data ingestion patterns using Machine Learning techniques. This project proposes a predictive ML-based anomaly detection framework capable of learning normal telecom traffic behavior from historical PM datasets and identifying abnormal deviations through forecasting-based deviation analysis.

Instead of relying solely on ML based XGBoost model, the project evaluates other advanced forecasting models including Prophet and LSTM to predict expected traffic behavior and automatically identify anomalies when actual traffic significantly deviates from predicted patterns. At conclusion the evaluation approaches using all these models have been compared to arrive at the best model to be used. The developed framework aims to improve operational reliability, enhance data quality assurance, and strengthen AI-driven telecom analytics capabilities within Dialog Axiata PLC.

2 Background and Motivation

Telecom operators continuously collect Key Performance Indicators (KPIs) from radio access networks including LTE and 5G infrastructures.

Typical PM metrics include:

- Downlink traffic volume
- Connected user counts
- User throughput
- Cell utilization
- Site availability
- Uplink traffic volume
- Throughput measurements of different QCI configurations

These KPIs are collected at hourly granularity across thousands of network entities.

Due to the scale and complexity of telecom infrastructure, ingestion failures can occur because of:

- Network connectivity interruptions
- Delayed file transfers
- Hardware failures
- Corrupted PM files
- Software configuration mismatches
- Missing reporting intervals
- Incomplete batches

Conventional rule-based detection systems require extensive manual tuning and often fail to adapt to dynamic traffic conditions.

Machine Learning forecasting models offer an improved approach by learning temporal patterns and automatically identifying deviations from expected behavior.

3 Problem Statement

The project focuses on developing an anomaly detection framework capable of identifying abnormal telecom traffic behavior at the ingestion stage of the data pipeline.

The system should:

- Learn normal telecom traffic patterns
- Forecast expected traffic values
- Compare predicted and actual traffic
- Detect abnormal deviations
- Identify missing or null datasets
- Generate anomaly files to do corrections

The project aims to improve the reliability of telecom analytics systems by preventing corrupted or incomplete data from propagating into downstream platforms.

4 Project Objectives

The primary objective of the project is to design and implement a Machine Learning-based anomaly detection system for telecom PM data ingestion pipelines.

This objective has been achieved by carefully splitting the overall goal into multiple sub objectives as listed below.

1. Develop a preprocessing pipeline for telecom PM data
2. Perform exploratory analysis of telecom traffic patterns
3. Engineer forecasting-related features
4. Implement multiple forecasting approaches
5. Compare ML and Deep Learning forecasting models
6. Detect anomalies using deviation analysis
7. Visualize anomalous behavior
8. Generate an output file with the identified list of anomalies
9. Build a reproducible ML experimentation framework

5 Literature Review

5.1 Telecom Data Forecasting Models

Telecom traffic forecasting has been widely studied due to its importance in network planning and operational management.

Traditional statistical forecasting methods have historically been used for telecom traffic prediction. Common approaches include:

- AutoRegressive Integrated Moving Average (ARIMA)
- Exponential Smoothing
- Seasonal Decomposition techniques

However, these models are effective for learning linear temporal relationships and periodic trends within structured datasets. However, modern telecom traffic generated by LTE and 5G networks often exhibits highly dynamic and nonlinear behavior due to variations in user mobility, application usage, network congestion, and time-dependent subscriber activity patterns. Further, telecom traffic also exhibits nonlinear fluctuations, bursty utilization patterns, and sudden operational variations that cannot always be accurately captured using conventional statistical forecasting methods.

Within this project, telecom forecasting was implemented using hourly KPI datasets collected from the network infrastructure of Dialog Axiata PLC. The primary KPI used for forecasting was `sum_dl_traffic_volume_gb`, representing the hourly downlink traffic volume generated at each telecom site. Historical traffic measurements collected from 2nd March 2026 to 4th March 2026 were used as the training dataset, while traffic data from 5th March 2026 was used for validation and anomaly detection.

The forecasting framework initially explored traditional time-series forecasting concepts but later transitioned toward Machine Learning and Deep Learning approaches due to the limitations observed

in statistical models when handling highly variable telecom traffic behavior. The implemented forecasting models include:

- XGBoost
- Prophet
- Long Short-Term Memory

The forecasting framework also incorporated extensive feature engineering techniques to improve predictive performance. Generated features included hour-of-day information, day-of-week attributes, lag-based traffic values, rolling traffic averages, connected user counts, and throughput-related KPI indicators.

Experimental observations demonstrated that Machine Learning forecasting models were more suitable for telecom KPI prediction than purely statistical approaches. Among the evaluated methods, XGBoost achieved the best overall balance between forecasting accuracy, computational efficiency, memory utilization, and operational stability.

5.2 Machine Learning-Based Forecasting

Machine Learning-based forecasting techniques have become increasingly popular in telecom analytics because of their ability to model nonlinear relationships and process large-scale operational datasets. Unlike traditional statistical forecasting methods, Machine Learning models can automatically learn complex interactions between multiple variables and improve prediction accuracy through feature engineering.

Tree-based ensemble learning algorithms such as XGBoost have gained significant attention in time-series forecasting research due to their strong predictive capabilities and robustness when handling structured tabular datasets.

XGBoost provides several advantages for telecom forecasting applications:

- Ability to model nonlinear relationships
- Strong support for engineered features
- High computational efficiency
- Scalability for large telecom datasets
- Improved forecasting accuracy
- Built-in regularization to reduce overfitting

Within this project, the XGBoost implementation utilized several engineered telecom traffic features including:

- Hour-of-day traffic behavior
- Day-of-week patterns
- Previous hour traffic (lag1)
- Previous day same-hour traffic (lag24)

These features enabled the model to learn recurring telecom traffic behavior and capture short-term traffic fluctuations more effectively than conventional statistical methods.

The XGBoost model was trained separately for each telecom site to learn site-specific traffic patterns. The generated forecasts were later compared with actual observed traffic measurements to identify abnormal deviations and potential data ingestion anomalies.

Results obtained during experimentation demonstrated that XGBoost consistently achieved superior forecasting accuracy while maintaining relatively low memory usage and faster execution times compared to Deep Learning approaches.

5.3 Deep Learning Forecasting

Deep Learning approaches have become increasingly important for sequential forecasting problems due to their ability to learn temporal dependencies and nonlinear sequential behavior from historical datasets. In telecom environments, traffic behavior often exhibits sequential dependencies where current traffic values are strongly influenced by previous observations.

One of the most widely used Deep Learning architectures for time-series forecasting is the Long Short-Term Memory network. LSTM models are a specialized form of Recurrent Neural Networks (RNNs) designed to overcome short-term memory limitations present in conventional RNN architectures.

LSTM networks provide several advantages for telecom forecasting:

- Ability to learn sequential traffic dependencies
- Capability to capture long-term temporal behavior
- Effective modeling of nonlinear traffic fluctuations
- Improved learning of recurring traffic sequences

Within this project, the LSTM model was implemented to forecast hourly telecom traffic using historical downlink traffic measurements. Sequential input windows were generated using historical traffic samples, and the network learned temporal dependencies between consecutive traffic observations.

Although LSTM demonstrated the ability to model telecom traffic sequences effectively, several operational limitations were observed during implementation. The model required significantly higher computational resources and memory usage compared to XGBoost and Prophet. Training multiple LSTM models for large numbers of telecom sites also introduced runtime stability challenges within the Google Colab environment.

Despite these challenges, LSTM forecasting provided valuable experimental comparison results within the proposed anomaly detection framework.

5.4 Prophet and NeuralProphet

Prophet is a forecasting framework originally developed by Meta for practical business time-series forecasting applications. Prophet is designed to handle trend analysis, seasonality decomposition, and missing data while requiring relatively low configuration effort.

The Prophet model decomposes time-series data into:

- Trend components
- Seasonal components

- Holiday or event effects
- Residual error components

Within this project, Prophet was implemented as a benchmark forecasting model to evaluate telecom traffic prediction performance against Machine Learning and Deep Learning approaches.

Prophet provided several operational advantages:

- Simple implementation
- Fast training speed
- Low memory utilization
- Built-in seasonality modeling
- Stable execution within Google Colab

However, Prophet primarily focuses on trend and seasonality decomposition and may struggle to capture highly nonlinear telecom traffic behavior.

The project also experimentally evaluated NeuralProphet, which extends Prophet by incorporating neural network optimization and autoregressive learning capabilities.

NeuralProphet introduced several enhancements including:

- Autoregressive forecasting
- Neural network optimization
- Lag-based learning
- Improved nonlinear modeling

Although NeuralProphet demonstrated improved nonlinear learning capability compared to Prophet, it required significantly higher computational resources and introduced dependency management challenges during implementation and therefore could not generate results for comparison purposes.

5.5 Anomaly Detection in Telecom Systems

Anomaly detection plays a critical role in telecom operational analytics because abnormal traffic behavior may indicate network faults, ingestion failures, corrupted KPI files, missing PM records, or unexpected operational events.

Several anomaly detection approaches have been studied within telecom systems, including:

- Threshold-based detection
- Statistical deviation analysis
- Density estimation techniques
- Autoencoder reconstruction errors
- Forecasting residual analysis

Traditional threshold-based methods are simple to implement but often generate excessive false alarms due to the dynamic nature of telecom traffic behavior.

Within this project, forecasting residual deviation analysis was selected as the primary anomaly detection mechanism. The implemented approach compared forecasted traffic values generated by the ML models against actual observed traffic measurements.

Deviation percentages were calculated using forecast residuals, and anomalies were identified whenever traffic deviations exceeded predefined thresholds. Null traffic values within validation datasets were also treated as anomalies because they may indicate ingestion failures or missing KPI records.

The anomaly detection framework successfully identified abnormal telecom traffic behavior while reducing the limitations associated with static threshold-based monitoring systems. The integration of forecasting and anomaly detection enabled the system to dynamically adapt to varying traffic conditions across telecom sites operated by Dialog Axiata PLC.

6 Dataset Description

6.1 Dataset Source

The dataset used in this project consists of telecom PM KPI measurements collected from LTE network infrastructure.

The dataset contains hourly site-level KPI measurements.

6.2 Dataset Structure

Key columns include:

Column	Description
date	Measurement date
hour	Measurement hour
site_name	Telecom site identifier
sum_avg_connected_users	Average connected users
sum_dl_traffic_volume_gb	Total downlink traffic
sum_avg_dl_user_thp_qci_8_kbps_num	QCI 8 throughput numerator
sum_avg_dl_user_thp_qci_8_kbps_deno	QCI 8 throughput denominator
sum_avg_dl_user_thp_qci_9_kbps_num	QCI 9 throughput numerator
sum_avg_dl_user_thp_qci_9_kbps_deno	QCI 9 throughput denominator

The dataset is characterized by:

- Hourly granularity
- Multivariate time-series behavior
- Telecom traffic seasonality
- Temporal dependencies
- Large-scale operational measurements

7 System Architecture

7.1 Data Ingestion

Raw PM KPI files are ingested into the processing environment. Dialog Axiata PLC stores network performance data in Microsoft Fabric Environment and all the Engineers perform their analysis on the same platform. The anomaly detection planned through this project needs to run on the data set prior to be used by the users for further analysis.

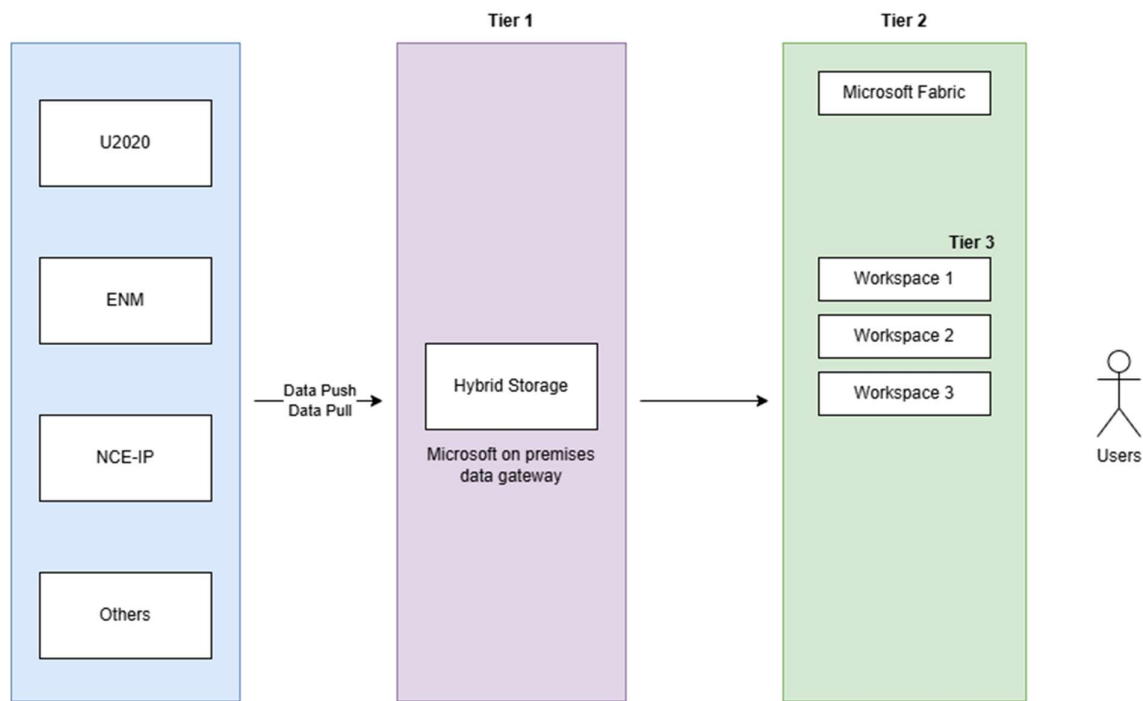


Figure 1: Dialog Axiata PLC Data Fetching Architecture

The proposed system architecture consists of the following stages/layers:

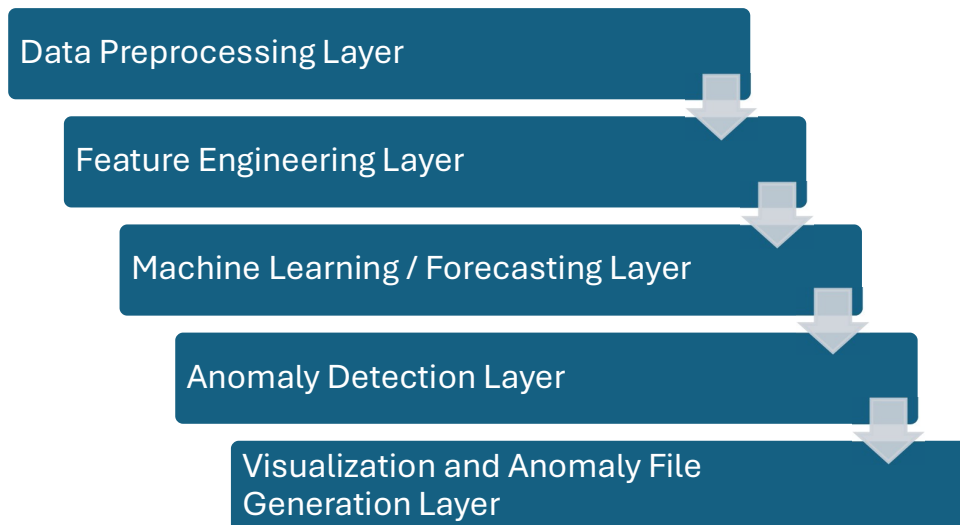


Figure 2: Key Layers of the Proposed System

7.2 Data Preprocessing Layer

The data preprocessing layer was designed to clean, standardize, and integrate network performance data collected from multiple telecom vendors, including Ericsson and Huawei systems. Initially, selected KPI columns such as connected users, downlink traffic volume, and throughput indicators were extracted from hourly PM datasets. The preprocessing workflow filtered records within a defined date range and data type conversions were performed to ensure numerical accuracy during aggregation and analysis. Additionally, site names were extracted from cell identifiers using string-splitting techniques, enabling site-level analysis. Finally, datasets from different vendors were merged into a unified structure for downstream processing and reporting.

7.3 Feature Engineering Layer

The feature engineering layer transformed the cleaned telecom KPI data into meaningful analytical features suitable for performance evaluation and predictive modeling. Aggregated features were generated at both hourly and daily site levels using grouping and summation operations. Key engineered features included total connected users, cumulative downlink traffic volume, and aggregated throughput numerator and denominator values for QCI 8 and QCI 9 services. Temporal attributes such as date and hour were retained to support time-based trend analysis and forecasting activities. Site-level identifiers were also engineered from cell names to simplify network-wide comparisons. These engineered features enabled efficient monitoring of sector performance and provided structured inputs for further machine learning and operational analytics tasks.

7.4 Machine Learning / Forecasting Layer

The forecasting layer was implemented to predict hourly downlink traffic volumes for telecom sites using multiple machine learning and time-series forecasting techniques. Four forecasting approaches were evaluated: XGBoost, Prophet and Long Short-Term Memory (LSTM) networks. Historical traffic data from previous days were used to train models, while future hourly traffic values were predicted for validation periods. Model performance was evaluated using MAE, RMSE, and R^2 metrics. This multi-model forecasting strategy enabled comparative analysis of predictive accuracy and supported proactive network capacity planning and performance optimization activities.

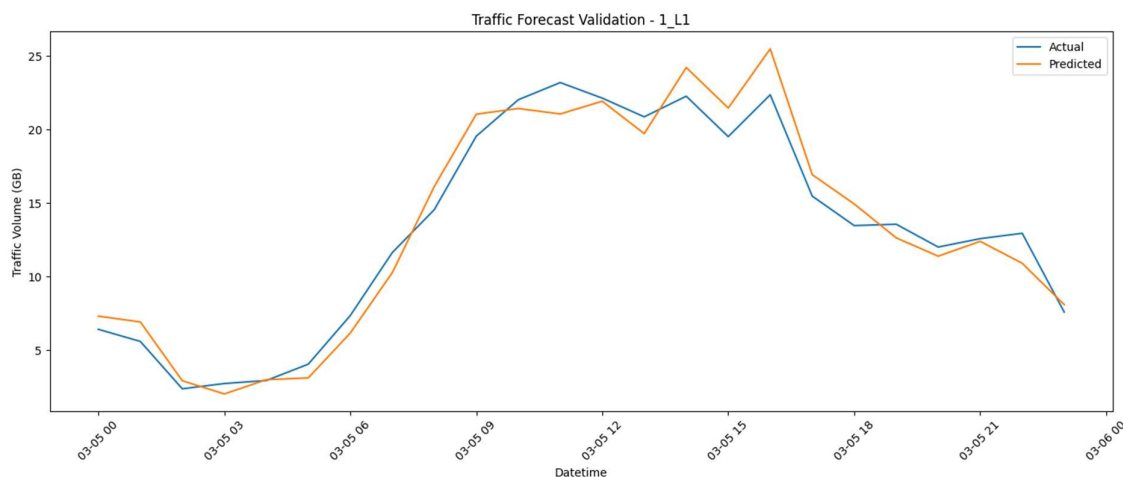


Figure 3: XGBoost based Machine Learning Actual vs Forecasted Daily Traffic Pattern

7.5 Anomaly Detection Layer

The anomaly detection layer identified unusual traffic behavior by comparing actual traffic measurements against machine learning forecasted values. After generating predictions using XGBoost, Prophet and LSTM models, deviation scores were calculated between predicted and actual traffic values. A threshold of 100% deviation was used to classify abnormal network behavior. The solution also detected null-value anomalies where expected traffic measurements were missing. For each telecom site, anomalies were filtered and consolidated into structured anomaly datasets containing timestamps, predicted traffic, actual traffic, deviation percentages, and anomaly flags. This automated detection process enabled early identification of network irregularities, traffic spikes, and potential operational issues requiring further investigation or corrective action.

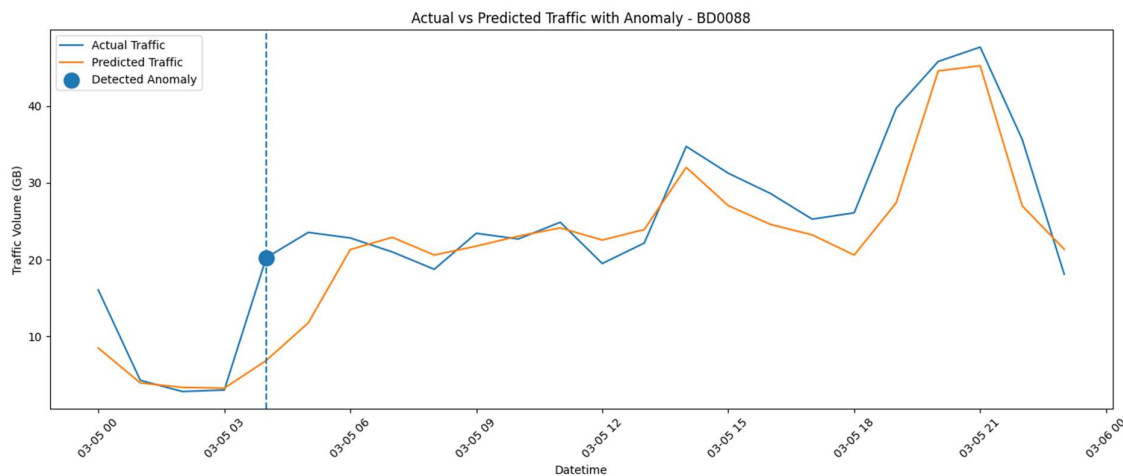


Figure 4:XGBoost based Identified Anomalous Data Point

7.6 Visualization and Anomaly File Generation Layer

The visualization and anomaly file generation layer provided graphical representations and structured reporting outputs for telecom traffic forecasting and anomaly analysis. Multiple visualization routines were implemented using Matplotlib to display hourly traffic trends, forecast validation comparisons, anomaly highlights, and model performance comparisons. Actual versus predicted traffic graphs were generated for selected sites to visually assess forecasting accuracy. Detected anomalies were emphasized using scatter plots and timestamp markers to improve interpretability. In addition, anomaly reports generated from each forecasting model were exported into CSV files for operational use and further analysis. Comparative bar charts for MAE, RMSE, and R^2 metrics were also created to evaluate overall model performance effectiveness.

8 Methodology

This project adopted a complete end-to-end machine learning methodology to forecast telecommunication network traffic and detect anomalies using hourly Performance Management (PM) data collected from telecom network infrastructure. The methodology integrates data preprocessing, feature engineering, machine learning forecasting, anomaly detection, visualization, and performance evaluation into a unified analytical workflow. The implementation was carried out using Python-based data science libraries including Pandas, NumPy, Scikit-learn, XGBoost, Prophet and Matplotlib. The

overall objective of the methodology was to predict downlink traffic behavior at telecom site level while automatically identifying abnormal traffic conditions that may indicate congestion, faults, outages, or unexpected network behavior.

8.1 Data Collection

The project utilized hourly telecom PM datasets containing site-level Key Performance Indicators (KPIs) related to network traffic and user activity. The dataset was loaded into a Python machine learning environment using the Pandas library. The source data included parameters such as date, hour, site name, average connected users, downlink traffic volume, and throughput indicators for different Quality of Service (QoS) classes including QCI 8 and QCI 9. These KPIs were selected because they directly influence network traffic behavior and user experience within mobile broadband networks.

The raw dataset was imported from CSV-based telecom reporting files into dataframes for analytical processing. Initial exploratory analysis was performed to verify column structures, data types, and sample records. The project also included data sorting operations to organize the records chronologically by site and timestamp. This preparation ensured that the forecasting models could properly learn temporal traffic patterns from historical observations. By centralizing telecom PM data into a structured machine learning environment, the methodology established a scalable foundation for forecasting and anomaly detection operations.

8.2 Datetime Construction

A dedicated datetime construction process was implemented to transform separate date and hour attributes into a unified timestamp feature suitable for time-series forecasting. Initially, the date column was converted into Python datetime format using Pandas datetime conversion functions. Subsequently, the hour field was converted into a time duration component and combined with the date field to generate a complete hourly timestamp.

This datetime feature became the primary temporal index used throughout the forecasting and anomaly detection pipeline. The generated timestamps enabled accurate chronological ordering of telecom traffic records and supported time-series model training processes. The dataset was then sorted by site name and datetime to preserve sequential ordering required for lag generation, rolling statistics, and deep learning sequence modeling.

The datetime construction process also improved visualization quality by enabling hourly trend analysis and timestamp-based plotting of network traffic patterns. Accurate timestamp representation is critical in telecom forecasting because traffic behavior typically exhibits strong hourly and daily seasonality patterns. Therefore, constructing a reliable datetime feature significantly enhanced the quality of downstream forecasting and anomaly detection activities.

8.3 Feature Engineering

Feature engineering played a major role in improving forecasting accuracy by generating additional predictive variables from the original telecom PM dataset. Several temporal and lag-based features were created to capture traffic dynamics and recurring network behavior patterns.

The methodology generated time-based features including hour of the day and day of the week. These variables enabled the forecasting models to learn daily traffic cycles and periodic network usage behavior. Telecom traffic typically varies significantly between peak and off-peak periods; therefore, incorporating temporal indicators improved the ability of the models to capture seasonality effects.

Lag-based features were also engineered to incorporate historical traffic dependency into the prediction process. Lag-1 and Lag-24 variables were created using previous hourly traffic values and traffic measurements from the same hour of the previous day. These lag variables allowed the machine learning models to learn short-term temporal dependencies and repetitive daily traffic patterns.

Additionally, rolling mean features were calculated using rolling windows across hourly traffic values. The rolling average smoothed short-term fluctuations and provided contextual trend information to the forecasting models. Other network KPI features such as connected users and throughput numerators and denominators were directly incorporated into the model input feature set.

For the LSTM deep learning implementation, sequence-based input structures were generated using sliding windows with a 24-hour lookback period. Traffic values were also normalized using MinMax scaling to improve neural network training stability and convergence. Overall, the feature engineering layer transformed raw telecom traffic data into a rich predictive dataset capable of supporting advanced machine learning and time-series forecasting models.

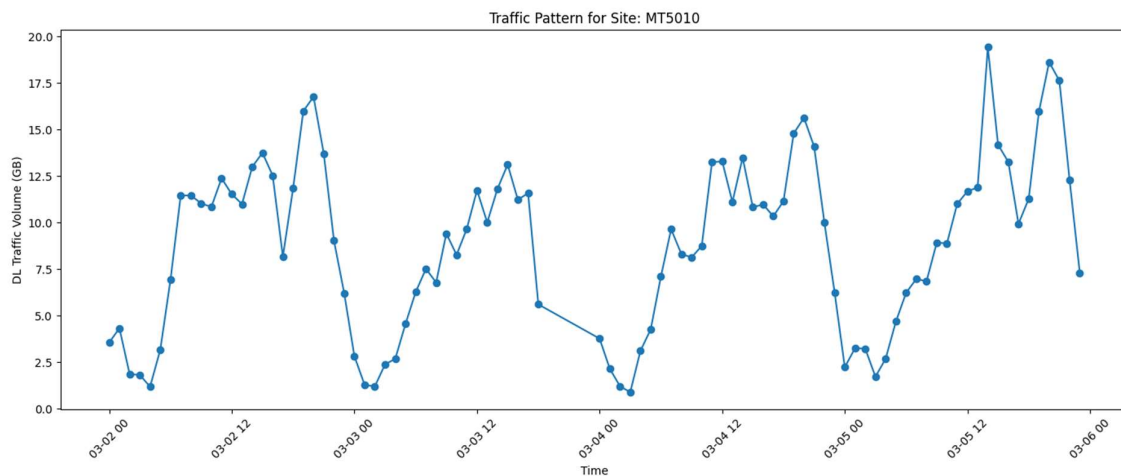


Figure 5: Sample Traffic Pattern of a Site

8.4 Training and Validation Split

The dataset was divided into training and validation periods to evaluate forecasting performance on unseen data. Historical traffic records from 2nd March to 4th March were used as the training dataset, while records from 5th March were reserved exclusively for validation and testing purposes. This chronological split preserved the temporal nature of the telecom traffic data and prevented future information leakage into the training process.

Rows containing missing values generated during lag feature creation were removed before model training. The methodology also implemented filtering conditions to skip weak or insufficiently populated telecom sites that did not contain enough historical observations for reliable forecasting.

For example, sites with fewer than 24 hourly training samples were excluded from certain forecasting procedures.

The validation dataset was used to compare actual traffic values against forecasted traffic values generated by the models. This evaluation strategy enabled objective assessment of forecasting accuracy using multiple statistical performance metrics including Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 score. These metrics provided quantitative insights into model prediction quality and forecasting reliability across different machine learning approaches.

8.5 Machine Learning / Forecasting

The forecasting layer implemented multiple machine learning and deep learning techniques to predict hourly telecom traffic volumes. Four forecasting approaches were evaluated: XGBoost, Prophet and Long Short-Term Memory (LSTM) neural networks.

The XGBoost model utilized engineered temporal, lag, and throughput features to perform supervised regression-based forecasting. Hyperparameters such as learning rate, tree depth, and number of estimators were configured to optimize prediction performance. Prophet and NeuralProphet models focused on learning seasonality and trend behavior within telecom traffic time series. Prophet-based implementations utilized daily seasonality configurations while excluding unnecessary weekly seasonality due to the short forecasting horizon.

The NeuralProphet implementation extended traditional Prophet functionality by integrating autoregressive neural network components capable of capturing more complex nonlinear traffic patterns. But due to computational limitations the output using this model was not generated. Meanwhile, the LSTM implementation leveraged recurrent neural network architecture to model sequential traffic dependencies using historical 24-hour traffic windows.

Each forecasting model generated hourly predictions for validation periods at site level. Predicted values were compared against actual observed traffic values to evaluate forecasting performance. Comparative analysis was then performed across all forecasting models using MAE, RMSE, and R^2 metrics. The methodology also included visualization routines that plotted actual versus predicted traffic behavior for selected telecom sites, enabling visual assessment of forecasting accuracy and temporal pattern learning.

8.6 Anomaly Detection

The anomaly detection methodology was implemented on top of the forecasting framework to identify abnormal telecom traffic behavior automatically. The core assumption was that significant deviations between actual traffic values and forecasted traffic values indicate potential anomalies within the network.

For each forecasting model, deviation scores were calculated using the relative difference between predicted traffic and actual observed traffic values. A predefined anomaly threshold of 100% deviation was applied to classify abnormal traffic behavior. If the deviation exceeded the threshold, the observation was flagged as a deviation anomaly.

The methodology also identified null anomalies by detecting missing traffic measurements within validation datasets. Combining deviation-based anomalies and null anomalies enabled the framework to capture both unexpected traffic fluctuations and missing KPI reporting scenarios.

All detected anomalies were consolidated into structured anomaly datasets containing timestamps, site identifiers, actual traffic, predicted traffic, deviation percentages, and anomaly flags. These anomaly reports were exported into CSV files for operational analysis and troubleshooting purposes. Visualization techniques were further applied to highlight anomaly points on traffic trend graphs using scatter plots and vertical timestamp markers. This enabled engineers and analysts to quickly identify abnormal network events and investigate potential operational issues affecting telecom site performance.

9 Comparison of Machine Learning Models

The project implemented and evaluated four different machine learning and time-series forecasting approaches to predict hourly downlink traffic behavior in telecommunication sites. The selected models were XGBoost, Prophet and Long Short-Term Memory (LSTM) neural networks. These models were chosen because they represent different categories of forecasting techniques, including traditional machine learning, statistical time-series forecasting, hybrid neural forecasting, and deep learning sequence modeling. The purpose of evaluating multiple forecasting models was to compare their prediction accuracy, temporal learning capability, computational efficiency, and suitability for telecom traffic forecasting applications.

The forecasting framework was designed to operate independently for each telecom site. Instead of creating a single global forecasting model, separate models were trained per site using site-specific traffic history and network KPI data. This approach enabled the models to learn localized traffic behavior unique to each telecom site, including variations in user density, traffic load, peak-hour utilization, and seasonal usage patterns. Telecom sites often demonstrate significantly different traffic characteristics depending on geographical location, subscriber behavior, and network infrastructure. Therefore, site-specific model training improved forecasting precision and reduced generalized prediction errors.

The machine learning workflow followed a structured methodology consisting of data preprocessing, datetime construction, feature engineering, model training, validation, forecasting, anomaly detection, and visualization. Historical telecom traffic data from 2nd March to 4th March were used for training purposes, while 5th March data were reserved for validation and performance testing. Model evaluation was performed using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and coefficient of determination (R^2) metrics. These performance indicators enabled quantitative comparison between forecasting models and facilitated selection of the most effective forecasting technique for telecom traffic prediction.

The project also incorporated anomaly detection mechanisms on top of the forecasting layer. Once traffic predictions were generated, deviations between actual and forecasted traffic values were calculated. Large deviations beyond predefined thresholds were classified as anomalies. This forecasting-driven anomaly detection methodology enabled proactive identification of abnormal

network behavior such as unexpected traffic spikes, network outages, congestion events, or missing KPI records.

In addition to prediction accuracy, the project also considered computational complexity, scalability, implementation flexibility, and memory management challenges associated with different forecasting approaches. Lightweight and memory-optimized implementations were developed for NeuralProphet and LSTM models to address computational limitations encountered during experimentation. Overall, the multi-model evaluation strategy provided a comprehensive understanding of the strengths and limitations of various forecasting technologies within telecom network analytics environments.

9.1 XGBoost-Based Forecasting Model

Extreme Gradient Boosting (XGBoost) is an advanced ensemble machine learning algorithm based on gradient boosting decision trees. It is widely recognized for its high predictive accuracy, computational efficiency, and strong performance on structured tabular datasets. In this project, XGBoost was implemented as the primary machine learning forecasting model for telecom traffic prediction. The model was trained independently for each telecom site using historical KPI data and engineered temporal features.

Unlike traditional statistical forecasting methods, XGBoost can learn highly nonlinear relationships between input variables and target traffic values. This capability is particularly valuable in telecommunications environments where traffic patterns are influenced by multiple interacting factors such as subscriber behavior, throughput conditions, connected users, and time-dependent traffic fluctuations.

9.1.1 Advantages

Several advantages made XGBoost highly suitable for this telecom forecasting project. First, the model demonstrated strong forecasting accuracy across most telecom sites. Its gradient boosting mechanism effectively minimized prediction errors by iteratively improving weak decision trees.

Second, XGBoost provided strong nonlinear learning capability. Telecom traffic behavior is inherently nonlinear due to fluctuating user activity patterns, changing network conditions, and dynamic service usage. The model successfully captured these nonlinear traffic variations using engineered lag and KPI features.

Third, XGBoost handled engineered features efficiently. Temporal variables such as hour of day, day of week, lag values, and rolling averages were incorporated directly into the model input feature set. The algorithm effectively learned complex relationships between these features and future traffic values.

Finally, XGBoost demonstrated efficient processing performance on structured telecom datasets. Compared to deep learning models, the training process required lower computational resources while still delivering high forecasting accuracy.

9.1.2 Model Configuration

The XGBoost implementation utilized the following hyperparameters during model training:

Parameter	Value
n_estimators	100
max_depth	4
learning_rate	0.1
random_state	42

The number of estimators controlled the number of boosting trees generated during training. A maximum depth of four was selected to balance model complexity and generalization capability. The learning rate parameter regulated boosting updates and prevented excessive overfitting. A fixed random state ensured reproducibility of forecasting experiments.

9.1.3 Forecasting Strategy

The forecasting strategy involved training a separate XGBoost regression model for each telecom site. Historical telecom traffic data were grouped by site name, and feature engineering techniques were applied to create predictive variables.

The model utilized several categories of features, including KPI-based features, temporal features, and historical lag features. KPI features included connected users and throughput indicators for QCI 8 and QCI 9 traffic classes. Temporal features included hour of day and day of week attributes. Lag-based features included previous-hour traffic values and traffic measurements from the same hour on the previous day. Rolling average statistics were also incorporated to capture short-term traffic trends.

Once trained, the model generated hourly traffic forecasts for validation periods. Actual and predicted traffic values were compared using performance evaluation metrics and visualization plots.

9.1.4 Performance

XGBoost achieved strong forecasting performance across most telecom sites evaluated in the project. The model consistently produced lower MAE and RMSE values while maintaining high R^2 scores relative to several alternative forecasting approaches. The ability to incorporate engineered KPI features and historical lag variables significantly improved forecasting precision.

The model also demonstrated robustness against short-term traffic fluctuations and changing traffic loads. Compared to statistical forecasting approaches such as Prophet, XGBoost exhibited superior nonlinear learning capability and improved adaptability to complex telecom traffic behavior. Overall, XGBoost emerged as one of the strongest forecasting approaches within the project.

9.2 Prophet Forecasting Model

Prophet is a time-series forecasting framework developed by Meta for handling business and operational forecasting tasks involving seasonality and trend analysis. In this project, Prophet was implemented to forecast telecom traffic behavior using historical hourly traffic measurements. The framework models traffic data as a combination of trend, seasonality, and residual components.

The Prophet model required input data in a specialized format consisting of datetime values represented as “ds” and target traffic values represented as “y”. The telecom traffic dataset was transformed accordingly before model training. Daily seasonality was enabled because telecom traffic exhibits clear hourly usage cycles throughout the day.

Prophet focuses primarily on modeling temporal behavior using decomposable time-series structures. The model automatically learns traffic trends, recurring seasonal behavior, and periodic temporal patterns.

Within the telecom environment, daily seasonality was especially important because subscriber activity changes significantly throughout the day. Peak traffic periods typically occur during business hours and evening periods, while lower traffic volumes occur during late-night hours. Prophet successfully modeled these repetitive temporal behaviors using its built-in seasonality framework.

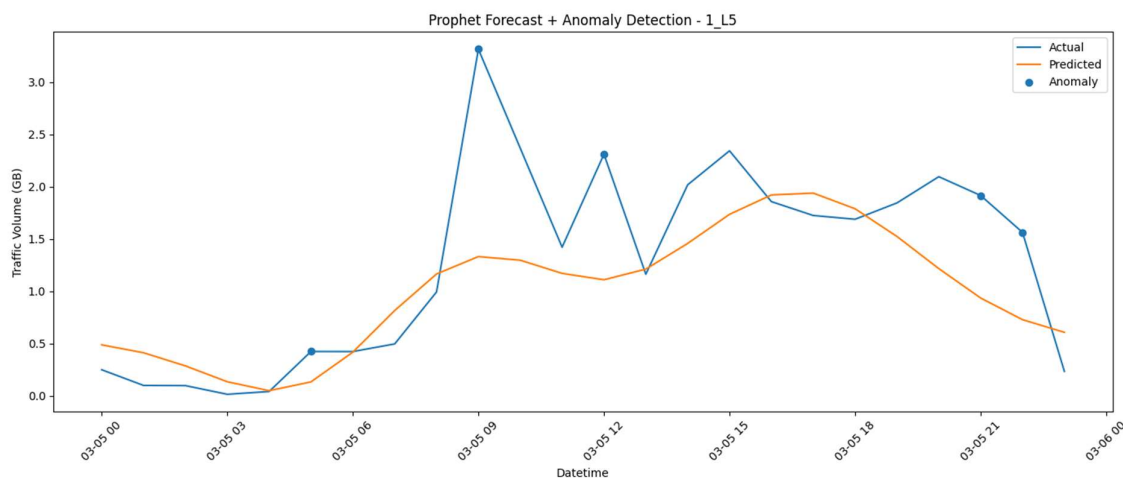


Figure 6: Prophet based detected Anomaly Sample

9.2.1 Advantages

One major advantage of Prophet was its simplicity and ease of implementation. The framework required relatively minimal feature engineering compared to machine learning approaches such as XGBoost.

Another advantage was interpretability. Prophet provides transparent forecasting structures based on identifiable trend and seasonal components, making it easier to understand prediction behavior.

The model also includes built-in seasonality handling capabilities, reducing the need for manually engineered seasonal variables. This made Prophet effective for learning periodic telecom traffic patterns.

9.2.2 Limitations

Despite its advantages, Prophet exhibited several limitations within the telecom forecasting environment. The model has limited support for multivariate feature integration compared to machine learning algorithms such as XGBoost. Consequently, important KPI variables could not be fully leveraged during forecasting.

Additionally, Prophet demonstrated lower nonlinear learning capability. Telecom traffic often exhibits highly nonlinear and dynamic behavior influenced by numerous operational conditions. Prophet's additive forecasting structure struggled to capture these complex nonlinear interactions effectively.

The model also showed reduced adaptability when traffic patterns changed abruptly or deviated significantly from historical seasonal behavior.

9.2.3 Performance

Prophet demonstrated reasonable forecasting performance for telecom traffic prediction. The model successfully captured daily seasonality and general traffic trends for many telecom sites. However, comparative evaluation results indicated that Prophet generally underperformed relative to XGBoost and some deep learning approaches.

The forecasting errors were higher in scenarios involving rapid traffic changes or highly irregular traffic behavior. Nevertheless, Prophet remained valuable as a lightweight and interpretable baseline forecasting framework for telecom analytics applications.

9.3 LSTM Forecasting Model

Long Short-Term Memory (LSTM) networks are specialized recurrent neural networks designed for sequential and time-dependent data processing. In this project, LSTM models were implemented to forecast telecom traffic using sequential learning techniques capable of capturing long-term temporal dependencies.

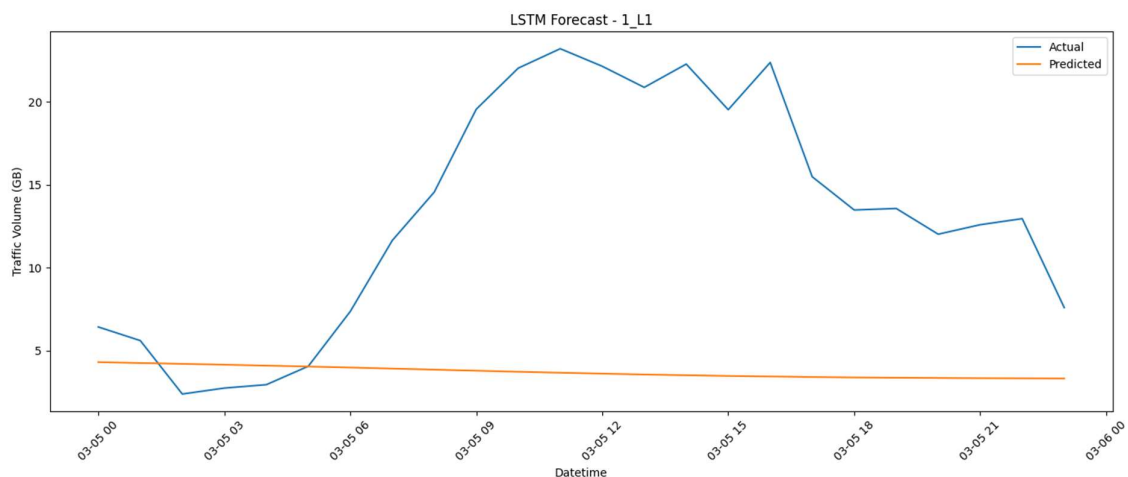


Figure 7: LSTM based Forecast for a Site

9.3.1 Sequence Learning

LSTM networks are particularly effective for time-series forecasting because they maintain memory states capable of learning long-term sequential relationships. Telecom traffic behavior often depends heavily on historical temporal patterns, making LSTM suitable for this forecasting task.

The implemented model used historical hourly traffic observations within a sliding window sequence structure. A 24-hour lookback window was applied so the model could learn daily traffic cycles and temporal continuity.

9.3.2 Model Architecture

The implemented LSTM architecture consisted of a single LSTM layer followed by a dense output layer. The LSTM layer learned temporal traffic dependencies from sequential input data, while the dense layer generated final traffic predictions.

Traffic values were normalized using MinMax scaling before model training to improve neural network convergence and stability. Sequential training samples were generated dynamically using rolling historical windows.

9.3.3 Hyperparameters

The LSTM model utilized the following configuration parameters:

Parameter	Value
Lookback Window	24
Epochs	20
Batch Size	16

The lookback window controlled the historical sequence length used during prediction. Epochs determined training iterations, while batch size regulated the number of training samples processed during optimization updates.

9.3.4 Advantages

LSTM demonstrated strong sequence learning capability and deep temporal understanding. The recurrent neural architecture effectively captured complex temporal dependencies and evolving traffic trends within telecom data.

The model also showed strong adaptability to dynamic traffic variations and irregular traffic fluctuations that are difficult to capture using statistical forecasting methods.

9.3.5 Limitations

The primary limitation of LSTM was its high computational requirement. Training deep learning models required significantly more processing time and memory resources compared to XGBoost and Prophet-based approaches.

The model also exhibited increased training complexity and dependency on larger volumes of historical data. Insufficient training data reduced forecasting stability and increased overfitting risk.

To manage resource limitations, lightweight memory-optimized LSTM implementations were introduced during experimentation, including reduced hidden units, fewer training epochs, and explicit memory cleanup procedures.

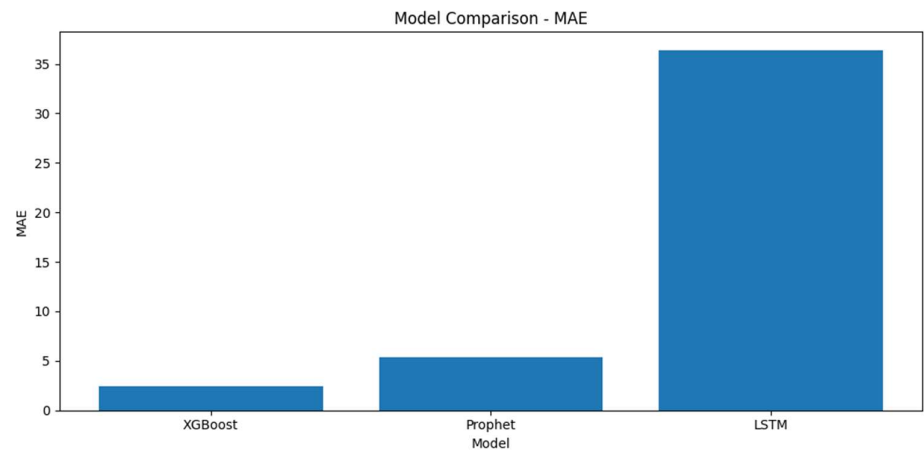


Figure 8: Mean Absolute Error Comparison of Models

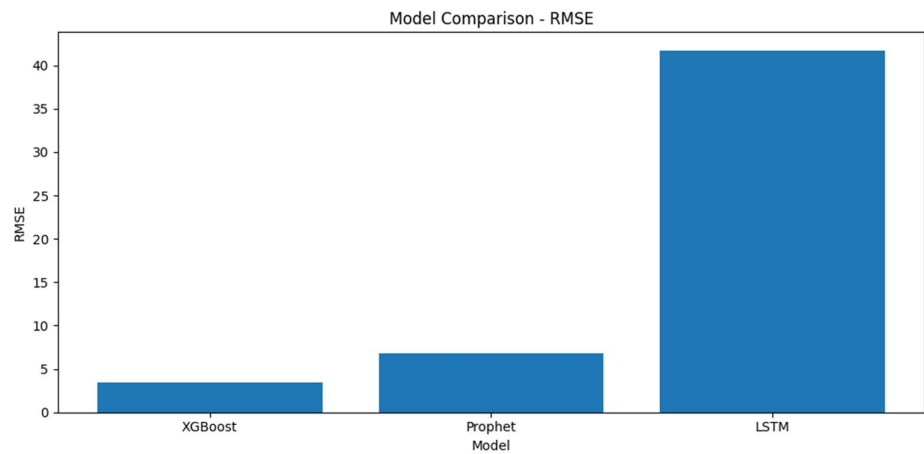


Figure 9: Root Mean Square Error Comparison of Models

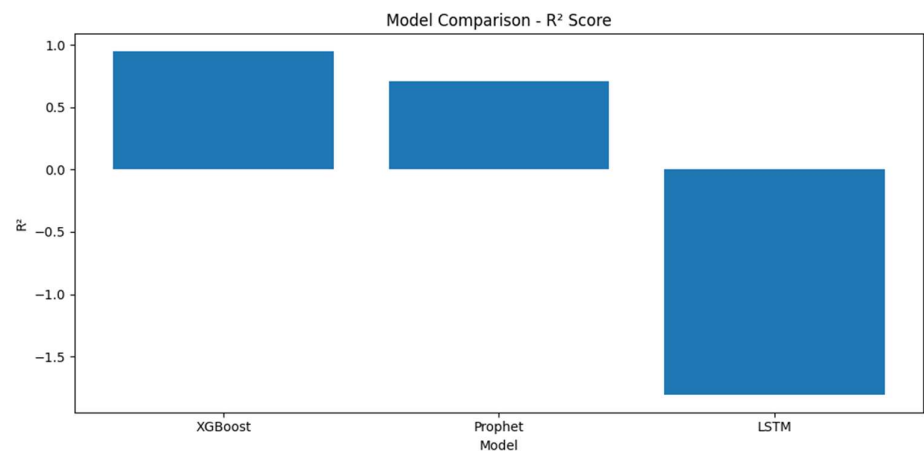


Figure 10: R Squared Comparison of Models

Based on the above indicators Best Performing Model for the analysis was XGBoost and it demonstrated following scores for respective areas.

Average R² Score : 0.9527

Average MAE : 2.4575

Average RMSE : 3.4274

10 Anomaly Detection Framework

The anomaly detection framework was developed as an intelligent monitoring layer integrated on top of the telecom traffic forecasting models. The primary objective of this framework was to automatically identify abnormal telecom traffic behavior by comparing real network traffic measurements against machine learning forecasted values. The framework was implemented across multiple forecasting approaches including XGBoost, Prophet and LSTM, enabling anomaly identification using predictive analytics rather than traditional static threshold monitoring. This methodology improved the ability to detect unexpected network events, data ingestion problems, and unusual traffic fluctuations within telecom infrastructure environments.

10.1 Detection Principle

The anomaly detection framework operated based on a forecast-versus-actual comparison strategy. For each telecom site, machine learning models generated predicted traffic values using historical KPI data and temporal traffic behavior. These forecasted values represented the expected normal traffic condition for a given hour. The system then compared the predicted traffic against the actual observed traffic recorded in the telecom PM dataset.

Under normal operating conditions, the actual traffic values were expected to remain reasonably close to the forecasted values. However, significant deviations between actual and predicted traffic indicated potential anomalies within the network. These deviations could represent abnormal traffic spikes, network congestion, missing data records, KPI corruption, or failures in the PM data ingestion process. By using predictive modeling as a baseline for normal behavior, the framework was capable of identifying anomalies dynamically rather than relying solely on fixed operational thresholds.

10.2 Deviation Calculation

The anomaly detection logic utilized a mathematical deviation formula to quantify the difference between actual and predicted traffic values. The deviation was calculated using the following equation:

$$\text{Deviation} = \text{Predicted} / |\text{Actual} - \text{Predicted}|$$

This formula measured the relative error between actual and forecasted traffic as a proportion of the predicted value. The use of relative deviation instead of absolute error enabled the framework to adapt effectively across telecom sites with different traffic volumes. For example, a traffic difference of 5 GB may be highly significant for a low-utilization site but negligible for a high-capacity site. Relative deviation normalized the anomaly measurement and ensured consistent detection sensitivity across all network sites.

The framework also included safeguards against divide-by-zero scenarios by filtering out prediction results where forecasted traffic values were equal to zero. This improved numerical stability and prevented invalid anomaly calculations during model evaluation and operational execution.

10.3 Threshold-Based Detection

The project implemented threshold-based anomaly classification using predefined deviation limits. Once deviation scores were calculated, records exceeding the specified thresholds were automatically flagged as anomalies. Three threshold levels were evaluated during experimentation: 100%, 50% and 30% but finally settled to 100% at the initial stage.

The 100% threshold represented a more conservative detection approach designed to identify only major traffic abnormalities. This configuration reduced false positive detections but risked missing smaller anomalies that could still represent operational issues. In contrast, the 30% and 50% thresholds provided a more sensitive detection mechanism capable of identifying moderate traffic deviations and subtle network abnormalities.

Threshold selection played an important role in balancing anomaly sensitivity and operational practicality. Lower thresholds increased detection sensitivity but also increased the probability of false alarms. Higher thresholds reduced false positives but potentially overlooked smaller anomalies. By evaluating multiple threshold levels, the framework enabled flexible anomaly detection tuning based on operational monitoring requirements and network management priorities.

10.4 Null Detection

In addition to deviation-based anomaly detection, the framework separately identified null-value anomalies within the telecom PM dataset. Null values represented missing or incomplete KPI measurements and were treated as ingestion anomalies.

The framework explicitly checked whether actual traffic values were null or unavailable during validation periods. Such scenarios may indicate failures in PM data generation, incomplete KPI collection, corrupted files, or ingestion pipeline interruptions. By separately flagging null anomalies, the system improved visibility into data quality problems that could negatively affect forecasting accuracy and operational analytics reliability.

This null detection capability was particularly important in telecom operational environments where missing PM batches can disrupt reporting systems and impair network performance analysis activities.

10.5 Operational Relevance

The anomaly detection framework provides significant operational value within telecom network management environments. Detected anomalies may represent several important operational conditions including missing PM batches, partial ingestion failures, corrupted KPI uploads, or sudden traffic abnormalities caused by unexpected subscriber activity or network faults.

For example, unusually low traffic values may indicate outages, transmission failures, or KPI collection problems, while unexpectedly high traffic values may represent congestion events, temporary demand spikes, or abnormal subscriber behavior. Similarly, null anomalies can signal ingestion pipeline interruptions or reporting failures that require immediate investigation.

The automated anomaly detection capability enables proactive network monitoring by identifying abnormal conditions earlier than traditional manual monitoring approaches. This supports faster troubleshooting, improved operational efficiency, and enhanced service reliability. Furthermore, the structured anomaly reports generated by the framework can assist telecom engineers in root-cause analysis, capacity management, and operational decision-making processes. Overall, the anomaly detection framework demonstrates how machine learning forecasting techniques can be effectively integrated into intelligent telecom network assurance systems.

11 Experimental Setup

The experimental setup for this project was designed to support the development, training, evaluation, and visualization of multiple machine learning and deep learning forecasting models for telecom traffic anomaly detection. The implementation environment combined cloud-based computational resources, Python-based machine learning libraries, time-series forecasting frameworks, and visualization tools to create a complete analytical pipeline capable of handling telecom KPI datasets and forecasting workloads. The experimental configuration also supported anomaly detection, comparative model analysis, and graphical visualization of telecom traffic behavior.

11.1 Development Environment

The entire implementation was developed using Python as the primary programming language due to its extensive ecosystem of machine learning, data science, and visualization libraries. Python provided flexible support for data preprocessing, feature engineering, forecasting model development, anomaly detection, and performance evaluation.

Google Colab was selected as the notebook execution environment because it offers cloud-based computational resources, integrated package management, and simplified experimentation workflows. The notebook-based development approach enabled iterative experimentation, rapid model testing, and interactive visualization of telecom traffic patterns. Google Colab also simplified dependency installation for advanced frameworks such as NeuralProphet and TensorFlow.

Several machine learning and forecasting libraries were integrated into the experimental workflow. The Scikit-learn library was used extensively for model evaluation metrics including Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 score calculations. Scikit-learn utilities were also used for preprocessing functions such as MinMax scaling within the LSTM implementation.

The XGBoost framework was used to implement gradient boosting regression models for telecom traffic forecasting. Prophet and NeuralProphet libraries were incorporated for time-series forecasting experiments involving seasonal traffic learning and autoregressive temporal modeling. TensorFlow and Keras libraries were used to implement LSTM-based deep learning sequence forecasting models.

Matplotlib was utilized throughout the project for traffic visualization, anomaly highlighting, forecasting comparisons, and comparative model analysis. Visualization routines generated line graphs, scatter plots, and bar charts to improve interpretability of forecasting behavior and anomaly detection results.

11.2 Hardware

The experiments were executed using cloud-based notebook resources available through Google Colab. This cloud execution environment provided access to scalable CPU and memory resources suitable for machine learning experimentation. Since the project evaluated multiple forecasting approaches independently across telecom sites, computational efficiency and memory management became important considerations during implementation.

The XGBoost and Prophet models executed relatively efficiently using standard CPU resources because these models operate primarily on structured tabular datasets and statistical forecasting methods. However, the NeuralProphet and LSTM deep learning implementations required significantly higher computational resources due to iterative neural optimization and sequence-based processing.

Several memory optimization strategies were introduced within the implementation to address computational limitations encountered during experimentation. For NeuralProphet, reduced lag windows, smaller epoch counts, limited site processing, and explicit garbage collection mechanisms were implemented to minimize memory consumption. Similarly, the LSTM implementation used lightweight neural architectures with reduced hidden units and controlled training epochs to improve execution stability within the cloud environment. TensorFlow session cleanup and graph reset functions were also applied after model training to release memory resources and prevent notebook crashes.

11.3 Evaluation Metrics

The forecasting models were evaluated using three widely accepted regression performance metrics: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 score. These metrics enabled quantitative comparison of forecasting accuracy across different machine learning approaches and telecom sites.

Mean Absolute Error (MAE) measures the average magnitude of forecasting errors between predicted traffic values and actual observed traffic values. MAE provides a direct interpretation of average prediction deviation and is less sensitive to extreme outliers compared to squared-error metrics. Lower MAE values indicate higher forecasting accuracy.

Root Mean Squared Error (RMSE) measures the square root of the average squared prediction errors. RMSE penalizes larger forecasting errors more heavily, making it useful for identifying models that produce occasional large deviations. Lower RMSE values represent better model performance.

The R^2 score, also known as the coefficient of determination, measures the proportion of traffic variation explained by the forecasting model. Higher R^2 values indicate stronger goodness-of-fit and better predictive capability.

These metrics collectively provided a comprehensive evaluation framework for assessing forecasting reliability and model effectiveness across telecom traffic prediction tasks.

12 Results and Evaluation

The experimental results demonstrated that machine learning and deep learning forecasting models can effectively predict telecom traffic behavior and support anomaly detection within telecom KPI ingestion systems. Forecasting performance varied across telecom sites due to differences in traffic volume, subscriber activity, and traffic variability. However, several consistent observations emerged from the comparative evaluation process.

12.1 Forecasting Accuracy Evaluation

Among the evaluated models, XGBoost generally achieved the strongest overall forecasting performance. The model consistently produced lower MAE and RMSE values while maintaining high R^2 scores across most telecom sites. Its ability to process engineered KPI features, temporal variables, and lag-based traffic history enabled highly accurate traffic prediction.

Prophet demonstrated stable forecasting capability for telecom sites with clear seasonal traffic patterns. The framework effectively captured recurring daily traffic cycles and general traffic trends. However, Prophet struggled with highly nonlinear traffic fluctuations and abrupt traffic changes.

NeuralProphet improved forecasting capability beyond standard Prophet by incorporating autoregressive neural learning components. The model demonstrated stronger nonlinear temporal learning and improved adaptability to changing traffic behavior. Nevertheless, computational overhead and dependency complexity were higher compared to simpler forecasting approaches and therefore could not arrive at a conclusive scoring.

The LSTM implementation demonstrated strong sequence-learning capability and effective modeling of long-term temporal dependencies. The recurrent neural architecture successfully learned evolving traffic patterns from historical sequences. However, forecasting performance depended heavily on data quality, sequence length, and training stability.

12.2 Anomaly Detection Results

The anomaly detection framework successfully identified several categories of abnormal telecom behavior. Large traffic spikes were detected when actual traffic values exceeded forecasted traffic by substantial margins. Such spikes may represent temporary congestion events, abnormal subscriber activity, or unexpected traffic surges.

The framework also identified sudden traffic drops where observed traffic fell significantly below expected forecasted values. These conditions may indicate outages, transmission failures, KPI collection problems, or partial service disruptions.

Missing traffic values were separately detected through null anomaly identification logic. This capability enabled identification of ingestion failures, incomplete PM batch uploads, or corrupted KPI reporting processes.

Overall, the forecasting-based anomaly detection approach demonstrated strong practical applicability for telecom operational monitoring and ingestion assurance systems.

12.3 Visual Analysis

Visualization played a major role in interpreting forecasting behavior and anomaly detection outcomes. Actual versus predicted traffic plots clearly illustrated the forecasting accuracy achieved by different models.

Anomalous intervals appeared as visible deviation spikes between actual and forecasted traffic curves. Scatter markers and vertical timestamp indicators highlighted detected anomalies directly on traffic trend graphs, enabling rapid visual identification of abnormal traffic conditions.

The visualization outputs significantly improved interpretability of machine learning behavior and supported intuitive understanding of telecom traffic dynamics.

13 Comparative Analysis

Based on forecasting performance, computational efficiency, implementation simplicity, and operational stability, XGBoost generally emerged as the best-performing forecasting model within this project.

The model provided an effective balance between forecasting accuracy and execution efficiency while maintaining relatively simple implementation requirements. XGBoost also demonstrated strong robustness across different telecom sites and traffic behaviors. Its ability to process engineered telecom KPI features contributed significantly to improved forecasting reliability.

The experimental results demonstrated that tree-based machine learning models are highly effective for telecom KPI forecasting tasks. Telecom traffic datasets contain rich engineered features including throughput indicators, connected user counts, lag variables, and rolling traffic statistics.

Tree-based models such as XGBoost can efficiently process these structured telecom features and learn nonlinear traffic relationships without requiring extremely large datasets or complex neural architectures. Consequently, machine learning approaches offer strong practical applicability for telecom traffic analytics and anomaly detection systems.

14 Visualization and Interpretation

Several visualization techniques were implemented to improve interpretability of telecom traffic forecasting and anomaly detection results. Visualization outputs enabled intuitive analysis of traffic trends, forecasting accuracy, and anomalous behavior across telecom sites.

14.1 Traffic Pattern Visualization

Hourly telecom traffic plots revealed important traffic characteristics including daily seasonality, peak-hour traffic behavior, and site-specific utilization patterns. Traffic volumes typically increased during business and evening hours while declining during late-night periods. Different telecom sites exhibited unique traffic signatures depending on user density and network demand conditions.

14.2 Prediction Comparison

Actual and predicted traffic values were plotted together on time-series graphs to evaluate forecasting quality visually. Close alignment between predicted and actual curves indicated strong forecasting accuracy, while visible gaps represented prediction errors or anomalous intervals.

14.3 Deviation Visualization

Deviation percentages between actual and forecasted traffic were also analyzed to evaluate anomaly severity. Large deviation spikes corresponded to abnormal traffic behavior and assisted in identifying operational issues requiring further investigation.

14.4 Anomaly Highlighting

Detected anomalies were highlighted using scatter markers and timestamp indicators directly on traffic trend plots. This visualization strategy enabled engineers and analysts to rapidly identify abnormal telecom events and assess forecasting reliability visually. Overall, the visualization framework improved transparency, interpretability, and operational usability of the forecasting system.

15 Discussion

The project demonstrates that machine learning forecasting models can significantly improve telecom ingestion anomaly detection compared to traditional rule-based monitoring systems. Conventional threshold-based monitoring approaches often struggle to adapt to dynamic telecom traffic behavior, site variability, and changing seasonal traffic patterns.

Machine learning models overcome these limitations by learning traffic behavior adaptively from historical KPI data. Forecasting models dynamically estimate expected traffic conditions and identify anomalies based on deviations from learned normal behavior rather than relying exclusively on static operational thresholds. This improves anomaly detection flexibility and reduces the limitations associated with manually configured monitoring rules.

The project also highlights the practical applicability of AI-driven analytics within telecom operational environments. Forecasting-based anomaly detection can support proactive monitoring, faster fault identification, improved operational efficiency, and enhanced service reliability. The integration of machine learning into telecom KPI ingestion systems demonstrates the growing importance of intelligent analytics for modern telecommunications infrastructure management.

16 Limitations

Despite promising results, several limitations were identified during the project implementation and experimentation process.

- Limited Training Duration

The forecasting models were trained using only a few days of historical telecom traffic data. Although sufficient for proof-of-concept experimentation, longer historical datasets would improve forecasting robustness, seasonality learning, and anomaly detection reliability.

- Limited Label Availability

The project primarily relied on unsupervised anomaly detection using forecast deviations because true labeled anomaly datasets were limited. Availability of validated anomaly labels would enable more comprehensive evaluation of detection precision and recall performance.

- Computational Constraints

Deep learning approaches such as NeuralProphet and LSTM required substantially higher computational resources compared to XGBoost and Prophet. Memory limitations and execution overhead constrained large-scale experimentation and required lightweight optimization strategies.

- Threshold Dependency

Although forecasting-based anomaly detection improves adaptability, the deviation-based anomaly classification process still depends on threshold selection. Different threshold values affect detection sensitivity and false-positive rates. Further research into adaptive thresholding techniques may improve anomaly detection robustness in future implementations.

Annex 1 - Reproducibility Checklist

This section provides a reproducibility checklist to ensure that the forecasting and anomaly detection experiments presented in this research can be replicated consistently by future researchers, telecom engineers, or academic reviewers. The checklist summarizes the datasets, software environment, preprocessing steps, forecasting configurations, evaluation methodology, and anomaly detection settings used throughout the project implementation. Establishing reproducibility is important in machine learning research because it improves transparency, validation, reliability, and scientific credibility of experimental findings.

Category	Reproducibility Item	Status
Dataset	Telecom PM dataset loaded from CSV format	✓
Dataset	Hourly site-level KPI records used	✓
Dataset	Training period: 2nd March – 4th March	✓
Dataset	Validation period: 5th March	✓
Preprocessing	Datetime feature construction implemented	✓
Preprocessing	Site-wise sorting performed	✓
Preprocessing	Missing values removed before training	✓
Feature Engineering	Hour feature generated	✓
Feature Engineering	Day-of-week feature generated	✓
Feature Engineering	Lag-1 feature implemented	✓
Feature Engineering	Lag-24 feature implemented	✓
Feature Engineering	Rolling mean feature implemented	✓
Development Environment	Python programming language used	✓
Development Environment	Google Colab notebook environment used	✓
Libraries	Pandas library used	✓
Libraries	NumPy library used	✓
Libraries	Matplotlib library used	✓
Libraries	Scikit-learn library used	✓
Libraries	XGBoost library used	✓
Libraries	Prophet library used	✓
Libraries	NeuralProphet library used	✓
Libraries	TensorFlow/Keras library used	✓
Forecasting Model	XGBoost model implemented	✓
Forecasting Model	Prophet model implemented	✓
Forecasting Model	NeuralProphet model implemented	✓
Forecasting Model	LSTM model implemented	✓
XGBoost Configuration	n_estimators = 100	✓
XGBoost Configuration	max_depth = 4	✓
XGBoost Configuration	learning_rate = 0.1	✓
XGBoost Configuration	random_state = 42	✓
Prophet Configuration	daily_seasonality enabled	✓
Prophet Configuration	weekly_seasonality disabled	✓
NeuralProphet Configuration	n_lags configured	✓
NeuralProphet Configuration	epochs configured	✓
NeuralProphet Configuration	learning_rate configured	✓

LSTM Configuration	Lookback window = 24	✓
LSTM Configuration	Batch size = 16	✓
LSTM Configuration	Sequential architecture implemented	✓
LSTM Configuration	MinMaxScaler normalization applied	✓
Evaluation Metrics	MAE evaluation implemented	✓
Evaluation Metrics	RMSE evaluation implemented	✓
Evaluation Metrics	R ² evaluation implemented	✓
Anomaly Detection	Forecast-versus-actual comparison used	✓
Anomaly Detection	Relative deviation calculation implemented	✓
Anomaly Detection	Threshold-based anomaly detection implemented	✓
Anomaly Detection	Null anomaly detection implemented	✓
Threshold Testing	30% threshold evaluated	✓
Threshold Testing	50% threshold evaluated	✓
Visualization	Actual vs predicted traffic plots generated	✓
Visualization	Anomaly highlighting implemented	✓
Visualization	Traffic trend visualization implemented	✓
Visualization	Comparative model evaluation charts generated	✓
Output Generation	CSV anomaly reports exported	✓
Output Generation	Forecast validation graphs generated	✓
Model Evaluation	Site-level forecasting evaluation performed	✓
Experiment Control	Fixed random_state used for reproducibility	✓
Computational Optimization	Memory optimization implemented for deep learning models	✓
Computational Optimization	TensorFlow session cleanup implemented	✓

Reproducibility Notes:

To reproduce the experiments successfully, researchers should ensure that the telecom KPI dataset contains the required hourly traffic and throughput fields used within the forecasting pipeline. The dataset should include timestamp-related fields, site identifiers, connected user metrics, throughput numerator and denominator KPIs, and downlink traffic volume measurements.

The experiments should preferably be executed within a cloud-based notebook environment such as Google Colab because the implementation depends on several machine learning and deep learning libraries requiring controlled dependency management.

For consistent forecasting behavior, the same training-validation split strategy, feature engineering methodology, and hyperparameter configurations should be preserved during replication experiments. Additionally, anomaly thresholds should remain unchanged unless adaptive threshold tuning is intentionally being investigated as part of future research extensions.